

Lab3 report

曹熠坤 2022110790

1. 程序功能

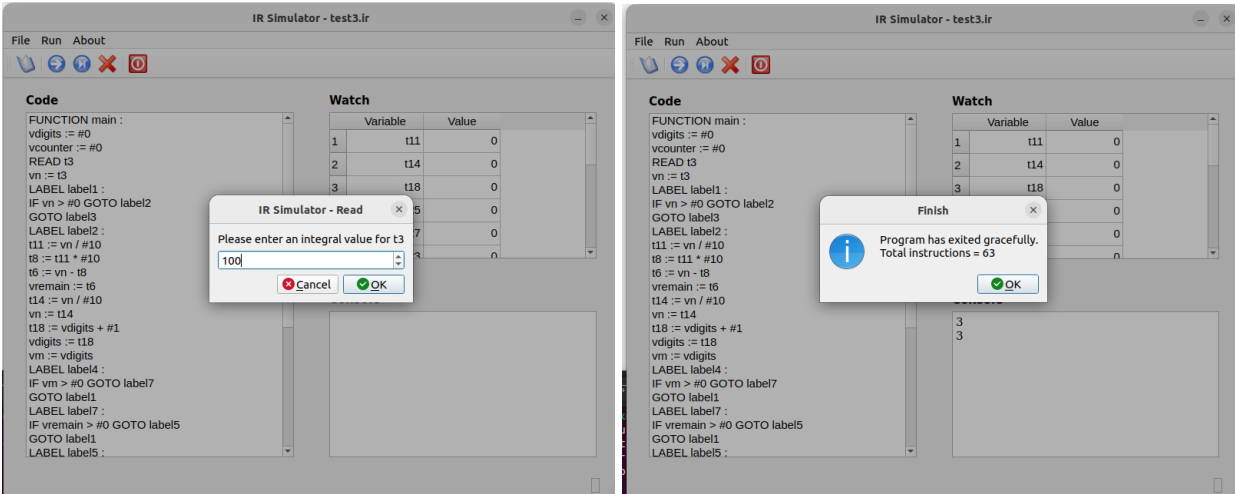
根据之前实验的词法分析和语法分析我们已经构造除了语法树和符号表，此基础上我们对此语法树的节点自上而下地进行递归调用，对每个语法成分进行翻译，并向最终结果中添加这个语法成分所对应的中间代码。本次完成了必做和选做的全部内容。

2. 实验结果

下面以给定的testcase_3为例演示实验结果：首先在实验目录中运行./parser testcase_3 test3.ir生成中间代码。生成的中间代码如下图：

```
1 FUNCTION main :
2 vdigits := #0
3 vcounter := #0
4 READ t3
5 vn := t3
6 LABEL label1 :
7 IF vn > #0 GOTO label2
8 GOTO label3
9 LABEL label2 :
10 t11 := vn / #10
11 t8 := t11 * #10
12 t6 := vn - t8
13 vremain := t6
14 t14 := vn / #10
15 vn := t14
16 t18 := vdigits + #1
17 vdigits := t18
18 vm := vdigits
19 LABEL label4 :
20 IF vm > #0 GOTO label7
21 GOTO label1
22 LABEL label7 :
23 IF vremain > #0 GOTO label5
24 GOTO label1
25 LABEL label5 :
26 t27 := vm * vremain
27 t25 := vcounter + t27
28 vcounter := t25
29 t30 := vm - #1
30 vm := t30
31 t33 := vremain - #1
32 vremain := t33
33 GOTO label4
34 GOTO label1
35 LABEL label3 :
36 WRITE vdigits
37 WRITE vcounter
38 RETURN #0
```

接着cd irsim切换到虚拟机的目录，执行python3.8 irsim.pyc激活小程序，然后加载test3.ir，点击运行，输入100，下面是程序的运行结果。



3. 编译方法

在./code目录下有Makefile文件，只需在linux环境下在该目录中运行指令即可自动编译，支持的指令如下：

```
make clean #清除parser以及所有编译生成的中间文件
make      #编译生成目标文件parser
make gdb  #自动在所有的编译指令加上-g选项，生成支持gdb调试功能的目标文件parser
```

4. 数据结构

在实验一和实验二的基础上，为表示和存储中间代码，还需新增如下数据结构：

- 1. 用于表示操作数类型的结构体Operand_及其指针Operand

```
typedef struct Operand_d Operand_;
typedef Operand_* Operand;
// 操作数（变量，临时变量，标记，函数）
struct Operand_d {
    enum {
        VARIABLE_OP, TEMP_VAR_OP, CONSTANT_OP, LABEL_OP,
        FUNCTION_OP, GET_ADDR_OP, GET_VAL_OP
    } kind;
    union {
        int no; // 临时变量的编号，标记的编号
        int value; // 常量的值
        char name[32]; // 函数以及变量的名字
        Operand opr; // 取地址和解引用指向的操作数
    };
    Type type; // 存放数组/结构体类型变量的类型
    Operand next; // 给argList链接操作数用
};
```

- 2. 用于表示中间代码结构的双向链表节点Intercode

```
typedef struct InterCode_d InterCode_;
typedef InterCode_* InterCode;
struct InterCode_d {
    enum {
        LABEL_IR, FUNC_IR, ASSIGN_IR, PLUS_IR, SUB_IR, MUL_IR,
        DIV_IR, TO_MEM_IR, GOTO_IR,
        IF_GOTO_IR, RETURN_IR, DEC_IR, ARG_IR, CALL_IR, PARAM_IR,
        READ_IR, WRITE_IR, NULL_IR
    } kind;
    // 操作数指针数组
    Operand ops[3];
    // 额外信息
    union {
        char relop[32]; // IF_GOTO_IR 指令的比较运算符
        int size;
    };
    // 前一条和后一条指令
    InterCode pre;
    InterCode next;
};
```

5. 翻译过程

进行中间代码生成时，自顶向下递归调用每一个语法结构的翻译函数。在每个翻译函数中，我们需要依据该语法结构对应的产生式对其进行解析，视具体情况，或通过递归调用其子过程的翻译函数生成中间代码并用临时变量保存其结果，或新建中间代码节点并为其各个字段填充内容，最后调用insertInterCode()函数逐步将生成的代码合并到一起。下面以表达式翻译函数translatorExp()中的一个分支（数组元素的赋值）为例进行讲解：

```

InterCode translateExp(Node* root, Operand place) {
    if (root->childNum == 3 && strcmp(root->children[1]->name, "ASSIGNOP") == 0) { //Exp -> Exp ASSIGNOP Exp
        else if (root->children[0]->childNum == 4 &&
            strcmp(root->children[0]->children[1]->name, "LB") == 0) { // Exp -> Exp    ASSIGNOP    Exp
                //                                //                                |-> Exp LB Exp RB
                //                                //                                a    [ i ]
                // tmp1应当是数组的地址, tmp2应当是一个整型, tmp3是所取元素的偏移量
                Operand tmp1 = newTemp();
                Operand tmp2 = newTemp();
                Operand tmp3 = newTemp();
                InterCode code1 = translateExp(root->children[0]->children[0], tmp1); // t1 := &a
                InterCode code2 = translateExp(root->children[0]->children[2], tmp2); // t2 := i
                int size = getSize(tmp1->type->array.elem);
                InterCode code3 = optimizeMULIR(tmp3, tmp2, getValue(size)); // t3 := t2 * size
                insertInterCode(code2, code1);
                insertInterCode(code3, code1);
                // tmp4存储所取元素的首地址
                Operand tmp4 = newTemp();
                InterCode code4 = optimizePLUSIR(tmp4, tmp1, tmp3); // t4 := t1 + t3
                insertInterCode(code4, code1);
                // tmp5存储的是右侧表达式的运算结果
                Operand tmp5 = newTemp();
                InterCode code5 = translateExp(root->children[2], tmp5); // t5 is the result of Exp
                InterCode code6 = (InterCode)malloc(sizeof(InterCode_)); // *t4 := t5
                code6->kind = TO_MEM_IR; /*
                code6->ops[0] = tmp4;
                code6->ops[1] = tmp5;
                insertInterCode(code5, code1);
                insertInterCode(code6, code1);
                // 将运算结果存回place
                if (place != NULL)
                    operandCpy(place, getVal(tmp4));
                return code1;
            }
        }
    }
}

```

进入该分支的条件即为：该`Exp`节点为赋值语句且其左值为数组引用。我们知道，数组元素赋值的关键是应计算该数组元素对应的地址。首先我们递归调用自身获得数组名对应的地址并存储到中间变量`tmp1`中，然后依然调用自身解析中括号中的`Exp`并将其值存储到`tmp2`中。为了确定偏移量(`tmp3`)，我们需要调用`getSize`函数获取`tmp1`所对应的数组的宽度，并生成一条乘法指令计算下标*宽度的值。该数组元素的地址(`tmp4`)即为基地址`tmp1`+偏移量`tmp3`，为此需要生成一条加法指令`tmp4 := tmp1 + tmp3`。我们还需将赋值号右侧的表达式进行处理并将其值存入`tmp5`，然后生成一条语句`*tmp4 = tmp5`向数组元素的地址中填入表达式的值。在此过程中，可以注意到，每生成一条新的指令，都会调用`insertInterCode()`将其追加到`code1`后，这样这个数组元素赋值对应的`Exp`节点所对应的中间代码就全部被存储到双向链表`code1`中，最后将它返回给上层过程即可。

6. 实验总结

中间代码作为编译器前后端的桥梁，其结构设计直接影响优化质量和目标代码生成效率。通过本次实验，我对于中间代码的生成过程与实现方法有了进一步的认识，并且在小程序执行中间代码的过程中深化了对它的理解，并体会到了中间代码在编译系统中桥梁的作用。